What I Built and Why

My capstone project explained in my own words. Use this to prepare for the demo.

The short version

I'm building a system that looks at a bank's operations data and tries to answer one question: is the system in trouble right now? For my first version, I focused on one specific job. For each hour, the model decides whether it was a **critical incident** hour, based on the health numbers the operations team already tracks, like CPU, memory, database connections, error rates, and transfer success rates.

Why this problem

Banks move money through NEFT, IMPS, and RTGS all day. When a system gets overloaded, customers start failing transfers before anyone on the team notices. The usual monitoring tools show business numbers, logs, and infrastructure numbers on separate screens, so it takes time to connect them. I wanted to see whether a simple model could spot the warning pattern from the numbers alone.

What the data looks like

I have one main file for this version: **incident_reports.csv**. It has 168 rows, one per hour, covering October 20 to 26, 2025. Of those hours, 142 are normal and 26 are critical. In critical hours, CPU averages about 90% versus about 49% in normal hours, and NEFT success drops to about 87% versus about 98%.

I also have a minute-by-minute file, **infrastructure_metrics.csv**, with 10,080 rows for the same week. I checked it for missing values, duplicate timestamps, and gaps, and it was clean.

What I found (and why it matters)

Finding 1: The labels follow a simple rule. If I mark an hour critical whenever NEFT success is below 95%, I get every label right. The lowest normal NEFT value is 96%, and the highest critical value is 89.8%. That means a model that uses NEFT will look almost perfect, but that score doesn't prove it learned anything. This is why I use the rule as my baseline, and why I asked my instructor how the labels were made.

Finding 2: The two files disagree. For 142 of the 168 hours, the incident file and the minute-level file match exactly. For the 26 critical hours, they don't. The incident file says NEFT is around 87% and CPU around 90%. The minute-level file shows NEFT around 98% and CPU mostly in the 35 to 45% range at those same hours. I don't know yet which one is correct, so I'm building v1 only on the incident file and documenting the conflict.

How I am approaching it

I kept the scope small on purpose, based on my instructor's feedback. I'm doing these steps in order:

1. **Baseline first.** The rule "critical if NEFT below 95%" is my starting point. It doesn't learn anything, so any model has to beat it to be useful.

2. **Time-based split.** I train on the first 5 days (120 hours, 17 critical) and test on the last 2 days (48 hours, 9 critical). I don't shuffle the data, because the hours are a time series. Shuffling would let the model see the future.

3. **One model.** A logistic regression on standardized features. I run it twice: once with NEFT included, and once without it. The second run is the important one, because it tests whether CPU, memory, connection pool, and error rate can identify critical hours on their own.

4. **Metrics.** I report precision, recall, and F1, plus a confusion matrix. I don't rely on accuracy, because critical hours are a small minority. With only 9 critical hours in the test set, one miss changes the score a lot, so I'll say that openly.

What the results show

Same-hour detection is perfect, and that is expected. A single rule on NEFT (or any other KPI) gets all 48 test hours right, and so does a logistic regression with or without NEFT. Because the labels separate that cleanly, this result shows the labels are rule-based, not that the model learned a pattern.

Early warning is the real test, and it is weak. Using the previous hour's KPIs to predict whether the next hour is critical, the best model catches 3 of the 9 critical test hours, with 11 false alarms (precision 0.21, recall 0.33, F1 0.26). The simple previous-hour NEFT rule catches none. Critical hours start abruptly, so the previous hour usually looks normal.

That is an honest negative result. The data doesn't support reliable early warning, and I report that rather than tuning until the numbers look better.

What is still open

I still need the instructor's answer on how the labels were created and which file is authoritative. The results above are for the incident file only.

What I am not doing in version 1

The original proposal included log clustering with NLP, a RAG assistant that answers questions about incidents, a FastAPI backend, and Docker. Those are stretch goals. I'd rather finish one evaluated model with honest results than leave several half-built pieces.

Possible questions and short answers

Why is the score so high? Because the labels look rule-based on NEFT. I show the rule matches the labels, and I report the model without NEFT as the real test.

Why not use all the minute-level data? The labels only exist at the hourly level, and the minute file conflicts with the labels at the critical hours. Using it would mean guessing labels.

What would you do next? Confirm the label source, test whether other KPIs carry the signal, and if there's time, add minute-level anomaly flags and a simple alert view in Streamlit.

Why is early warning so weak? Critical hours begin suddenly, so the previous hour's KPIs don't carry a warning sign in this data.

What did you learn? That the data you're given can look clean and still contradict itself, and that a simple baseline can beat a fancy-looking result. I'd rather report that honestly than hide it.